

Punto 4.9 - Compiled Queries

Práctica

Ejercicio: Añadir Compiled Queries al repositorio del proyecto AceriaData. Compilar las consultas más frecuentes. Comparar el tiempo de ejecución con las consultas no compiladas. Analizar el SQL generado por ambas técnicas.

Contexto del proyecto: En el punto 4.8 se estudiaron las Split Queries, incluyendo el producto cartesiano y la configuración del comportamiento por defecto. En este punto se profundiza en las Compiled Queries, que permiten reutilizar el plan de ejecución de las consultas más frecuentes. Esta técnica se usará en el punto 4.10 para la paginación eficiente.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear la clase de consultas compiladas

Crear el archivo src/AceriaData.Infrastructure/Repositories/OrdenConsultasCompiladas.cs:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public static class OrdenConsultasCompiladas
{
    public static readonly Func<AceriaDbContext, string, List<OrdenFabricacion>> ObtenerPorEstado =
        EF.CompileQuery((AceriaDbContext context, string estado) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => o.Estado == estado)
                .OrderBy(o => o.FechaCreacion)
                .ToList());

    public static readonly Func<AceriaDbContext, string, List<OrdenFabricacion>> ObtenerPorCliente =
        EF.CompileQuery((AceriaDbContext context, string cliente) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => o.Cliente == cliente)
                .OrderBy(o => o.FechaCreacion)
                .ToList());

    public static readonly Func<AceriaDbContext, DateTime, DateTime, List<OrdenFabricacion>>
    ObtenerPorRangoDeFechas =
        EF.CompileQuery((AceriaDbContext context, DateTime desde, DateTime hasta) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
                .OrderBy(o => o.FechaCreacion)
                .ToList());

    public static readonly Func<AceriaDbContext, int, OrdenFabricacion?> ObtenerPorId =
        EF.CompileQuery((AceriaDbContext context, int id) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .FirstOrDefault(o => o.Id == id));

    public static readonly Func<AceriaDbContext, int> ContarOrdenes =
```

```

    EF.CompileQuery((AceriaDbContext context) =>
        context.OrdenesFabricacion.Count());
}

```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 2: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 3: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 5: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 7: public static class OrdenConsultasCompiladas → declara la clase estática.

Línea 9: public static readonly Func<AceriaDbContext, string, List> ObtenerPorEstado = → declara el campo estático.

Línea 10: EF.CompileQuery((AceriaDbContext context, string estado) => → compila la consulta.

Línea 11: context.OrdenesFabricacion → inicia la consulta.

Línea 12: .AsNoTracking() → aplica AsNoTracking.

Línea 13: .Where(o => o.Estado == estado) → filtra por estado.

Línea 14: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 15: .ToList()); → materializa la consulta.

Línea 17: public static readonly Func<AceriaDbContext, string, List> ObtenerPorCliente = → declara el campo estático.

Línea 18: EF.CompileQuery((AceriaDbContext context, string cliente) => → compila la consulta.

Línea 19: context.OrdenesFabricacion → inicia la consulta.

Línea 20: .AsNoTracking() → aplica AsNoTracking.

Línea 21: .Where(o => o.Cliente == cliente) → filtra por cliente.

Línea 22: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 23: .ToList()); → materializa la consulta.

Línea 25: public static readonly Func<AceriaDbContext, DateTime, DateTime, List> ObtenerPorRangoDeFechas = → declara el campo estático.

Línea 26: EF.CompileQuery((AceriaDbContext context, DateTime desde, DateTime hasta) => → compila la consulta.

Línea 27: context.OrdenesFabricacion → inicia la consulta.

Línea 28: .AsNoTracking() → aplica AsNoTracking.

Línea 29: .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta) → filtra por rango de fechas.

Línea 30: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 31: .ToList()); → materializa la consulta.

Línea 33: public static readonly Func<AceriaDbContext, int, OrdenFabricacion?> ObtenerPorId = → declara el campo estático.

Línea 34: EF.CompileQuery((AceriaDbContext context, int id) => → compila la consulta.

Línea 35: context.OrdenesFabricacion → inicia la consulta.

Línea 36: .AsNoTracking() → aplica AsNoTracking.

Línea 37: `.FirstOrDefault(o => o.Id == id));` → filtra por Id y devuelve la primera coincidencia.

Línea 39: `public static readonly Func<AceriaDbContext, int> ContarOrdenes =` → declara el campo estático.

Línea 40: `EF.CompileQuery((AceriaDbContext context) =>` → compila la consulta.

Línea 41: `context.OrdenesFabricacion.Count();` → cuenta las órdenes.

Error común: si se compila una consulta que usa un método personalizado, EF Core lanza una excepción al compilarla. Se debe usar expresiones traducibles.

Paso 3: Añadir los métodos de Compiled Queries a la interfaz del repositorio

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerPorEstadoCompilada(string estado);
    List<OrdenFabricacion> ObtenerPorEstadoNoCompilada(string estado);
    List<OrdenFabricacion> ObtenerPorClienteCompilada(string cliente);
    OrdenFabricacion? ObtenerPorIdCompilada(int id);
    int ContarOrdenesCompilada();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: `using AceriaData.Application.Dtos;` → importa los DTOs.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 4: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 6: `public interface IOrdenRepositorio` → declara la interfaz.

Línea 8: `// ... métodos existentes ...` → comentario que indica que se mantienen los métodos anteriores.

Línea 10: `List ObtenerPorEstadoCompilada(string estado);` → método que usa la consulta compilada.

Línea 11: `List ObtenerPorEstadoNoCompilada(string estado);` → método que usa la consulta no compilada.

Línea 12: `List ObtenerPorClienteCompilada(string cliente);` → método que usa la consulta compilada por cliente.

Línea 13: `OrdenFabricacion? ObtenerPorIdCompilada(int id);` → método que usa la consulta compilada por Id.

Línea 14: `int ContarOrdenesCompilada();` → método que usa la consulta compilada de conteo.

Línea 16: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 17: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

Paso 4: Implementar los métodos de Compiled Queries en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepository.cs:

```
public List<OrdenFabricacion> ObtenerPorEstadoCompilada(string estado)
{
    return OrdenConsultasCompiladas.ObtenerPorEstado(_context, estado);
}

public List<OrdenFabricacion> ObtenerPorEstadoNoCompilada(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorClienteCompilada(string cliente)
{
    return OrdenConsultasCompiladas.ObtenerPorCliente(_context, cliente);
}

public OrdenFabricacion? ObtenerPorIdCompilada(int id)
{
    return OrdenConsultasCompiladas.ObtenerPorId(_context, id);
}

public int ContarOrdenesCompilada()
{
    return OrdenConsultasCompiladas.ContarOrdenes(_context);
}
```

Línea 1: public List ObtenerPorEstadoCompilada(string estado) → declara el método.

Línea 3: return OrdenConsultasCompiladas.ObtenerPorEstado(_context, estado); → invoca la consulta compilada.

Línea 6: public List ObtenerPorEstadoNoCompilada(string estado) → declara el método.

Línea 8: return _context.OrdenesFabricacion → inicia la consulta.

Línea 9: .AsNoTracking() → aplica AsNoTracking.

Línea 10: .Where(o => o.Estado == estado) → filtra por estado.

Línea 11: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 12: .ToList(); → materializa la consulta.

Línea 15: public List ObtenerPorClienteCompilada(string cliente) → declara el método.

Línea 17: return OrdenConsultasCompiladas.ObtenerPorCliente(_context, cliente); → invoca la consulta compilada.

Línea 20: public OrdenFabricacion? ObtenerPorIdCompilada(int id) → declara el método.

Línea 22: return OrdenConsultasCompiladas.ObtenerPorId(_context, id); → invoca la consulta compilada.

Línea 25: public int ContarOrdenesCompilada() → declara el método.

Línea 27: return OrdenConsultasCompiladas.ContarOrdenes(_context); → invoca la consulta compilada.

Error común: si la consulta compilada se usa con un DbContext distinto al que se usó para compilarla, EF Core lanza una excepción. Se debe usar el mismo tipo de DbContext.

Paso 5: Crear el caso de uso de Compiled Queries

Crear el archivo src/AceriaData.Application/UseCases/CompiledQueriesUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class CompiledQueriesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public CompiledQueriesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== COMPILED QUERIES ===");

        DemostrarConsultaCompilada();
        DemostrarConsultaNoCompilada();
        CompararRendimiento();
    }

    private void DemostrarConsultaCompilada()
    {
        Console.WriteLine("\n--- Consulta compilada ---");

        var cronometro = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            var ordenes = _unidad.Ordenes.ObtenerPorEstadoCompilada("Pendiente");
        }
        cronometro.Stop();

        Console.WriteLine($"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarConsultaNoCompilada()
    {
        Console.WriteLine("\n--- Consulta no compilada ---");

        var cronometro = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            var ordenes = _unidad.Ordenes.ObtenerPorEstadoNoCompilada("Pendiente");
        }
        cronometro.Stop();

        Console.WriteLine($"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms");
    }

    private void CompararRendimiento()
    {
        Console.WriteLine("\n--- Comparación de rendimiento ---");

        var cronometroCompilada = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            _unidad.Ordenes.ObtenerPorEstadoCompilada("Pendiente");
        }
        cronometroCompilada.Stop();

        var cronometroNoCompilada = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            _unidad.Ordenes.ObtenerPorEstadoNoCompilada("Pendiente");
        }
        cronometroNoCompilada.Stop();

        Console.WriteLine($"Compilada: {cronometroCompilada.ElapsedMilliseconds} ms");
    }
}
```

```

        Console.WriteLine($"No compilada: {cronometroNoCompilada.ElapsedMilliseconds} ms");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class CompiledQueriesUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public CompiledQueriesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== COMPILED QUERIES ==="); → muestra la cabecera.

Línea 19: DemostrarConsultaCompilada(); → llama al método de consulta compilada.

Línea 20: DemostrarConsultaNoCompilada(); → llama al método de consulta no compilada.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 24: private void DemostrarConsultaCompilada() → declara el método.

Línea 26: Console.WriteLine("\n--- Consulta compilada ---"); → muestra la cabecera.

Línea 28: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 29: for (int i = 0; i < 100; i++) → repite cien veces.

Línea 31: var ordenes = _unidad Ordenes.ObtenerPorEstadoCompilada("Pendiente"); → ejecuta la consulta compilada.

Línea 33: cronometro.Stop(); → detiene el cronómetro.

Línea 35: Console.WriteLine(\$"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 38: private void DemostrarConsultaNoCompilada() → declara el método.

Línea 40: Console.WriteLine("\n--- Consulta no compilada ---"); → muestra la cabecera.

Línea 42: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 43: for (int i = 0; i < 100; i++) → repite cien veces.

Línea 45: var ordenes = _unidad Ordenes.ObtenerPorEstadoNoCompilada("Pendiente"); → ejecuta la consulta no compilada.

Línea 47: cronometro.Stop(); → detiene el cronómetro.

Línea 49: Console.WriteLine(\$"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 52: private void CompararRendimiento() → declara el método.

Línea 54: Console.WriteLine("\n--- Comparación de rendimiento ---"); → muestra la cabecera.

Línea 56: var cronometroCompilada = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 57: for (int i = 0; i < 100; i++) → repite cien veces.

Línea 59: _unidad Ordenes.ObtenerPorEstadoCompilada("Pendiente"); → ejecuta la consulta compilada.

Línea 61: cronometroCompilada.Stop(); → detiene el cronómetro.

Línea 63: `var cronometroNoCompilada = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 64: `for (int i = 0; i < 100; i++)` → repite cien veces.

Línea 66: `_unidad Ordenes.ObtenerPorEstadoNoCompilada("Pendiente");` → ejecuta la consulta no compilada.

Línea 68: `cronometroNoCompilada.Stop();` → detiene el cronómetro.

Línea 70: `Console.WriteLine($"Compilada: {cronometroCompilada.ElapsedMilliseconds} ms");` → muestra el tiempo de la consulta compilada.

Línea 71: `Console.WriteLine($"No compilada: {cronometroNoCompilada.ElapsedMilliseconds} ms");` → muestra el tiempo de la consulta no compilada.

Error común: si el bucle ejecuta la consulta cien veces con el mismo parámetro, EF Core puede cachear la consulta y los resultados. Se debe usar parámetros distintos para medir el coste de compilación.

| Paso 6: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<CompiledQueriesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 7: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<CompiledQueriesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 8: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... } → sexta orden.

Línea 15: }; → cierra la lista.

Línea 17: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 18: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

| Paso 9: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de las cien ejecuciones de la consulta compilada y de la consulta no compilada.

| Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```

=== COMPILED QUERIES ===

--- Consulta compilada ---
100 ejecuciones: 85 ms

--- Consulta no compilada ---
100 ejecuciones: 120 ms

--- Comparación de rendimiento ---
Compilada: 85 ms
No compilada: 120 ms

```

La primera sección muestra el tiempo de las cien ejecuciones de la consulta compilada. La segunda sección muestra el tiempo de las cien ejecuciones de la consulta no compilada. La tercera sección muestra la comparación.

Observaciones: la consulta compilada es más rápida porque el plan de ejecución se reutiliza. La consulta no compilada paga el coste de compilación en cada ejecución, aunque EF Core tiene una caché de consultas que reduce el coste. La diferencia es mayor cuando la consulta es compleja o cuando la caché de consultas se llena.

Paso 11: Diagnosticar un error común

Modificar el método `ObtenerPorEstadoCompilada` para compilar la consulta cada vez que se llama:

```

public List<OrdenFabricacion> ObtenerPorEstadoCompilada(string estado)
{
    var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string e) =>
        context OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Estado == e)
            .OrderBy(o => o.FechaCreacion)
            .ToList());

    return consultaCompilada(_context, estado);
}

```

Resultado esperado: el código funciona, pero la consulta se compila cada vez que se llama al método. El beneficio de la consulta compilada se pierde porque el coste de compilación se paga en cada llamada.

Solución: mover la compilación al campo estático de la clase `OrdenConsultasCompiladas`.

```

return OrdenConsultasCompiladas.ObtenerPorEstado(_context, estado);

```

Resultado esperado con la solución: la consulta se compila una sola vez y se reutiliza en todas las llamadas.

Errores comunes del ejercicio

Error	Causa	Solución
Compilación en cada llamada	Se compila la consulta dentro del método	Mover la compilación al campo estático
DbContext distinto	Se usa un DbContext distinto al de la compilación	Usar el mismo tipo de DbContext
Método personalizado	Se usa un método no traducible en la consulta	Usar expresiones traducibles
Parámetros incorrectos	Los parámetros no coinciden con los de la compilación	Verificar la firma del delegado
Resultado cacheado	Se usa el mismo parámetro en todas las ejecuciones	Usar parámetros distintos

Error	Causa	Solución
Compiled Query innecesaria	La consulta se ejecuta pocas veces	Usar consultas normales

Retos resueltos: Comparar Compiled Queries con y sin proyección

Reto: Crear dos consultas compiladas, una que devuelva entidades completas y otra que devuelva una proyección. Comparar el tiempo de ejecución de cien iteraciones.

Solución paso a paso:

Paso 1: Añadir las consultas compiladas a OrdenConsultasCompiladas:

```
public static readonly Func<AceriaDbContext, string, List<OrdenFabricacion>> ObtenerPorEstadoCompleta =
    EF.CompileQuery((AceriaDbContext context, string estado) =>
        context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Estado == estado)
            .OrderBy(o => o.FechaCreacion)
            .ToList());

public static readonly Func<AceriaDbContext, string, List<OrdenResumenDto>> ObtenerPorEstadoProyectada =
    EF.CompileQuery((AceriaDbContext context, string estado) =>
        context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Estado == estado)
            .OrderBy(o => o.FechaCreacion)
            .Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .ToList());
```

Línea 1: public static readonly Func<AceriaDbContext, string, List> ObtenerPorEstadoCompleta = → declara el campo estático.

Línea 2: EF.CompileQuery((AceriaDbContext context, string estado) => → compila la consulta.

Línea 3: context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .Where(o => o.Estado == estado) → filtra por estado.

Línea 6: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 7: .ToList(); → materializa la consulta.

Línea 9: public static readonly Func<AceriaDbContext, string, List> ObtenerPorEstadoProyectada = → declara el campo estático.

Línea 10: EF.CompileQuery((AceriaDbContext context, string estado) => → compila la consulta.

Línea 11: context.OrdenesFabricacion → inicia la consulta.

Línea 12: .AsNoTracking() → aplica AsNoTracking.

Línea 13: .Where(o => o.Estado == estado) → filtra por estado.

Línea 14: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 15: .Select(o => new OrdenResumenDto) → proyecta al DTO.

Línea 16: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 17: Cliente = o.Cliente, → asigna el cliente.

Línea 18: Estado = o.Estado, → asigna el estado.

Línea 19: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 20: }) → cierra la proyección.

Línea 21: .ToList(); → materializa la consulta.

Paso 2: Añadir los métodos a la interfaz IOrdenRepositorio:

```
List<OrdenResumenDto> ObtenerPorEstadoProyectadaCompilada(string estado);
```

Línea 1: List ObtenerPorEstadoProyectadaCompilada(string estado); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```
public List<OrdenResumenDto> ObtenerPorEstadoProyectadaCompilada(string estado)
{
    return OrdenConsultasCompiladas.ObtenerPorEstadoProyectada(_context, estado);
}
```

Línea 1: public List ObtenerPorEstadoProyectadaCompilada(string estado) → declara el método.

Línea 3: return OrdenConsultasCompiladas.ObtenerPorEstadoProyectada(_context, estado); → invoca la consulta compilada.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarProyectadaCompilada()
{
    Console.WriteLine("\n--- Consulta compilada con proyección ---");

    var cronometro = Stopwatch.StartNew();
    for (int i = 0; i < 100; i++)
    {
        _unidad.Ordenes.ObtenerPorEstadoProyectadaCompilada("Pendiente");
    }
    cronometro.Stop();

    Console.WriteLine($"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms");
}
```

Línea 1: private void DemostrarProyectadaCompilada() → declara el método.

Línea 3: Console.WriteLine("\n--- Consulta compilada con proyección ---"); → muestra la cabecera.

Línea 5: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 6: for (int i = 0; i < 100; i++) → repite cien veces.

Línea 8: _unidad.Ordenes.ObtenerPorEstadoProyectadaCompilada("Pendiente"); → ejecuta la consulta compilada con proyección.

Línea 10: cronometro.Stop(); → detiene el cronómetro.

Línea 12: Console.WriteLine(\$"100 ejecuciones: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarProyectadaCompilada();
```

| Paso 6: Ejecutar dotnet run y comparar los tiempos.

Resultado esperado: la consulta compilada con proyección es más rápida que la consulta compilada con entidades completas porque transfiere menos datos.

| Analogía final

Las Compiled Queries en una acería son como las plantillas de órdenes de búsqueda que el jefe de planta prepara una sola vez y reutiliza muchas veces. En lugar de redactar la orden desde cero cada vez, el jefe usa una plantilla que ya tiene las condiciones predefinidas y solo rellena los parámetros. La primera vez que se usa la plantilla, se tarda un poco en prepararla. Las siguientes veces, se tarda mucho menos porque la plantilla ya está lista. Las consultas no compiladas son como redactar la orden desde cero cada vez: se tarda más porque hay que pensar en todas las condiciones. La caché de consultas de EF Core es como un archivador donde el jefe guarda las plantillas que ya ha usado. Si la plantilla está en el archivador, la reutiliza. Si no está, la redacta y la guarda. Las Compiled Queries son como las plantillas que el jefe ha preparado con antelación y tiene siempre a mano. En una acería con mucho volumen, las plantillas ahorran mucho tiempo. En una acería pequeña, no tanto. Así funcionan las Compiled Queries en EF Core: se compilan una vez y se reutilizan muchas veces.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado la clase OrdenConsultasCompiladas con cinco consultas compiladas.

Añadido los métodos de Compiled Queries a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso CompiledQueriesUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones de consulta compilada y no compilada.

Comparado el rendimiento de ambas técnicas.

Diagnosticado el error de compilar la consulta en cada llamada.

Creado la consulta compilada con proyección y comparado su rendimiento.
